

Order-Independent Transparency Methods in 3DGS Rendering

He Zhu

Shanghai Jiao Tong University

Outline

- **Background**
 - What is 3DGS?
 - Tile-based 3DGS rendering pipeline.
 - Problem: why do we need OIT?
- OIT methods in 3DGS
 - Weighted Sum Rendering
 - Surfel Rendering
 - Stochastic Transparency

What is 3D Gaussian Splatting?



Images



3DGS
Differentiable
Pipeline

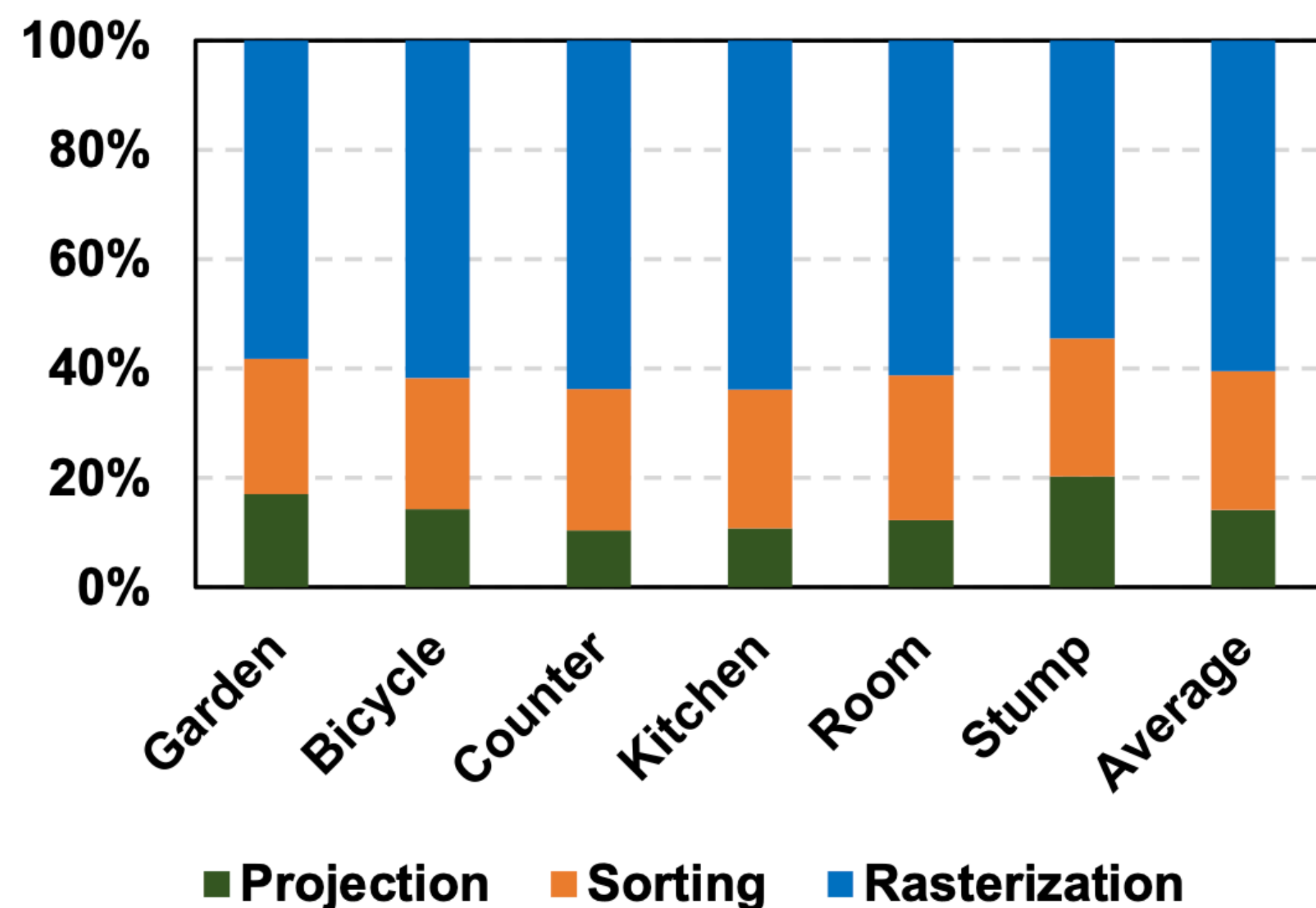
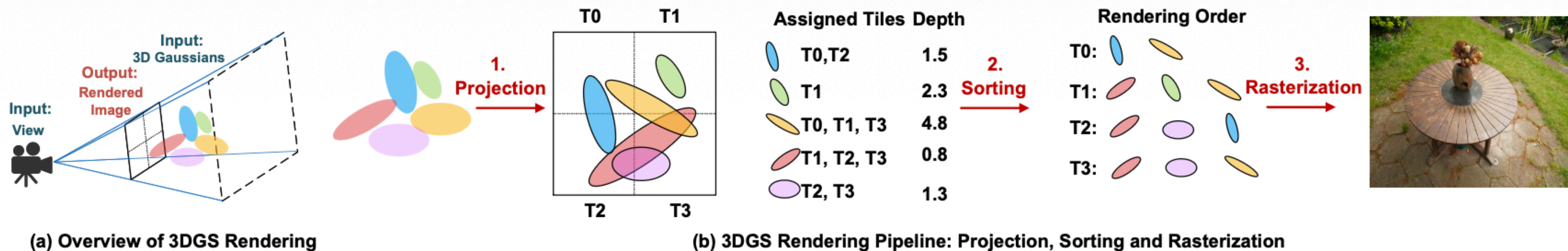


Point-based Representation



Photorealistic Rendering

3DGS Rendering Pipeline



Alpha blending in rasterization:

1. Init color $C = 0$ transmittance $T = 1$
2. Compute α for nearest Gaussian: $\alpha_i = o_i G_i(p)$
3. Blending: $C + = c_i \alpha_i T$
4. Update Transmittance: $T = T(1 - \alpha_i)$
5. If T is small enough, break; else, go to step 2.

Problem

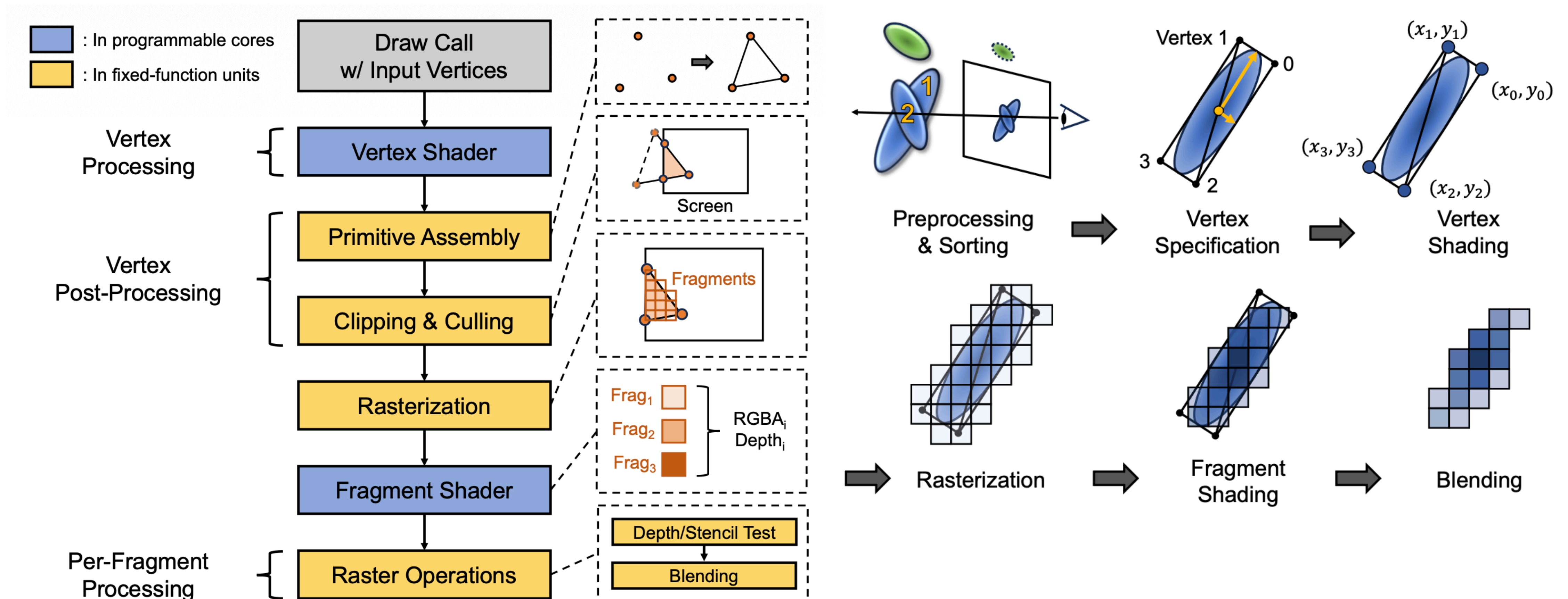
- Portability: Current rasterizer implementation is based on CUDA.
- Hardware Inefficiency: GPU graphic pipeline is not good at sorting and alpha-blending.

	Sorting	Blending	Portability
CUDA Pipeline	On SM, hardware feature utilized	On SM	Nvidia device
Compute Pipeline (compute Shader)	On SM, no hardware feature utilized	On SM	Not supported on some mobile device
Standard Graphic Pipeline (Vertex/Fragment Shader)	Not supported	On ROP that much fewer than SM	Generally supported

- Render Output Unit (ROP): hardware that execute blending, MSAA and output final pixels.

Problem

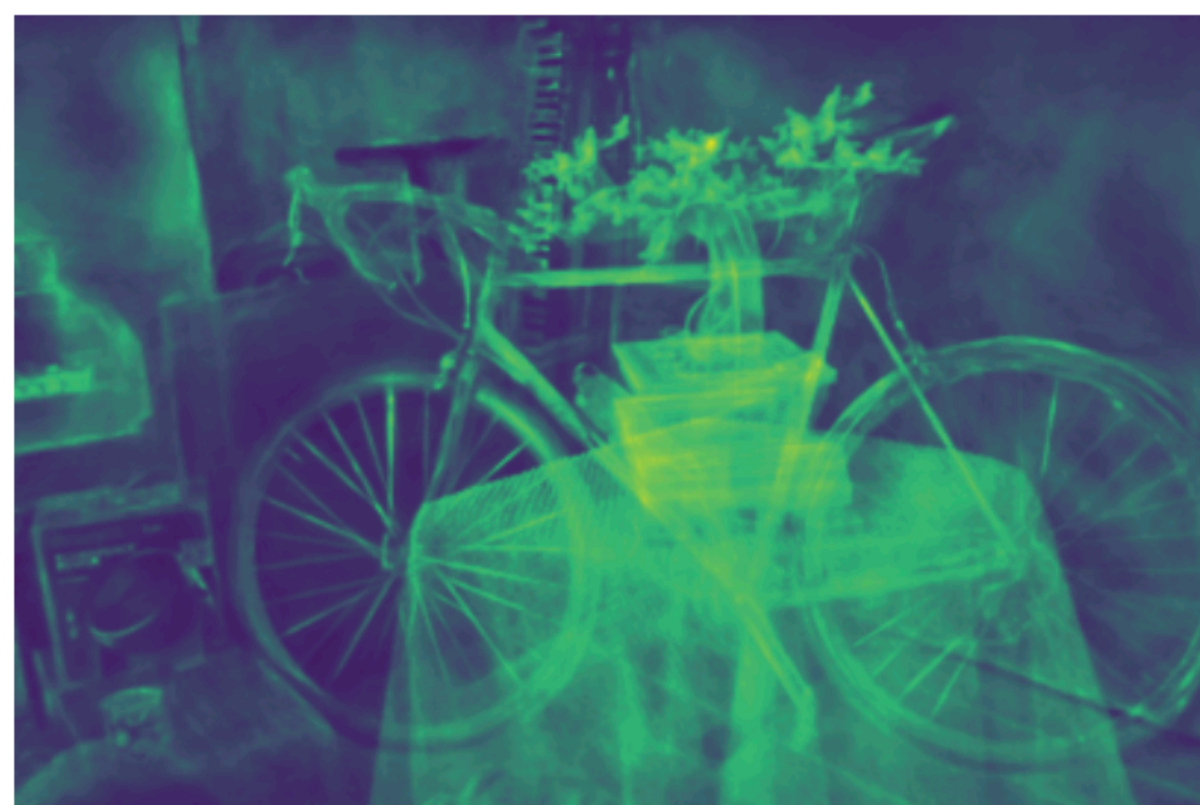
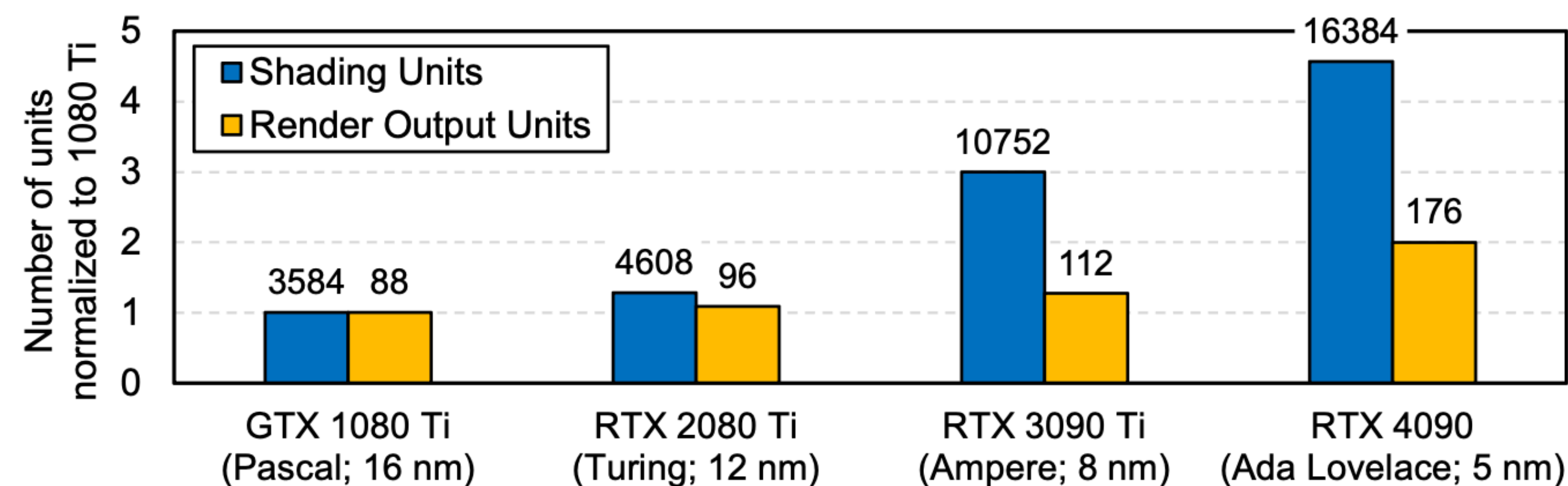
- What if we achieve 3DGS rendering via graphic pipeline?
 - New workflow: global sorting -> triangulation -> rasterization



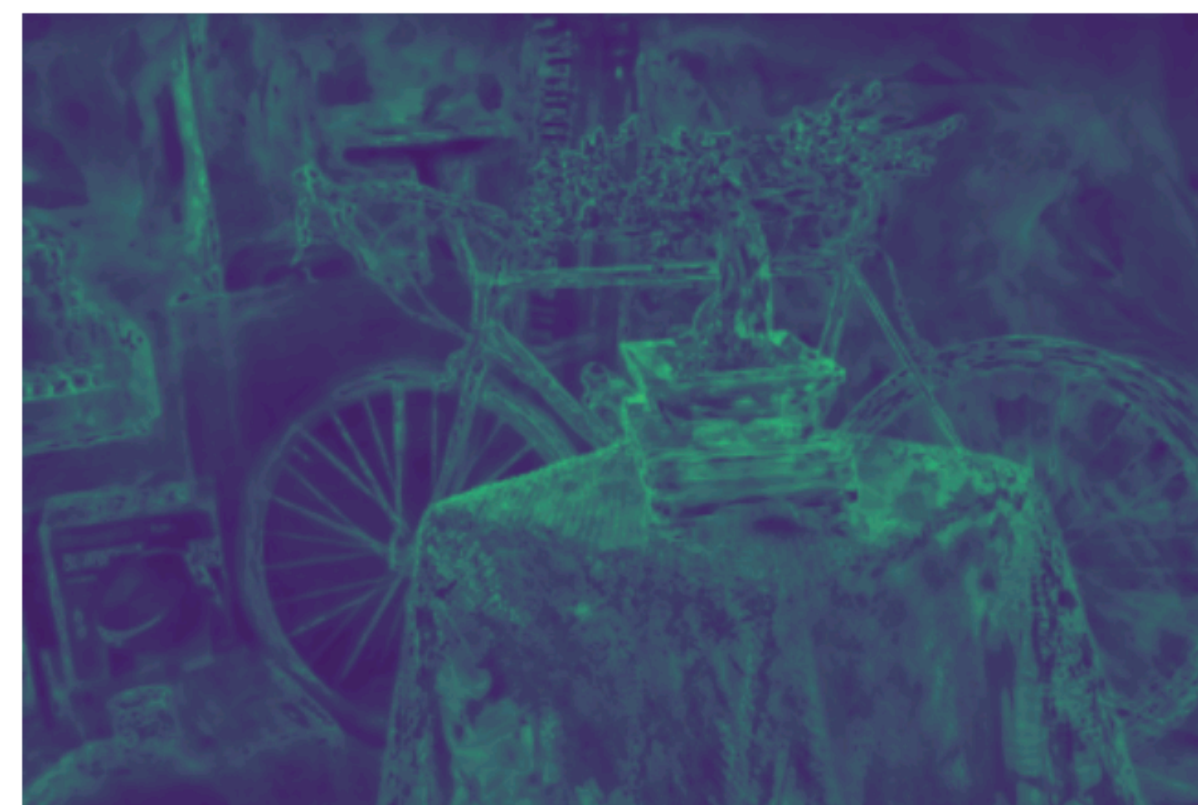
Problem

- What if we achieve 3DGS rendering via graphic pipeline?
 - New workflow: global sorting -> triangulation -> rasterization

	3DGS-CUDA	3DGS-OpenGL
T1000	65.08	100.28
RTX3090	8.14	32.15
RTX4090	5.60	20.70



(a) w/o early termination

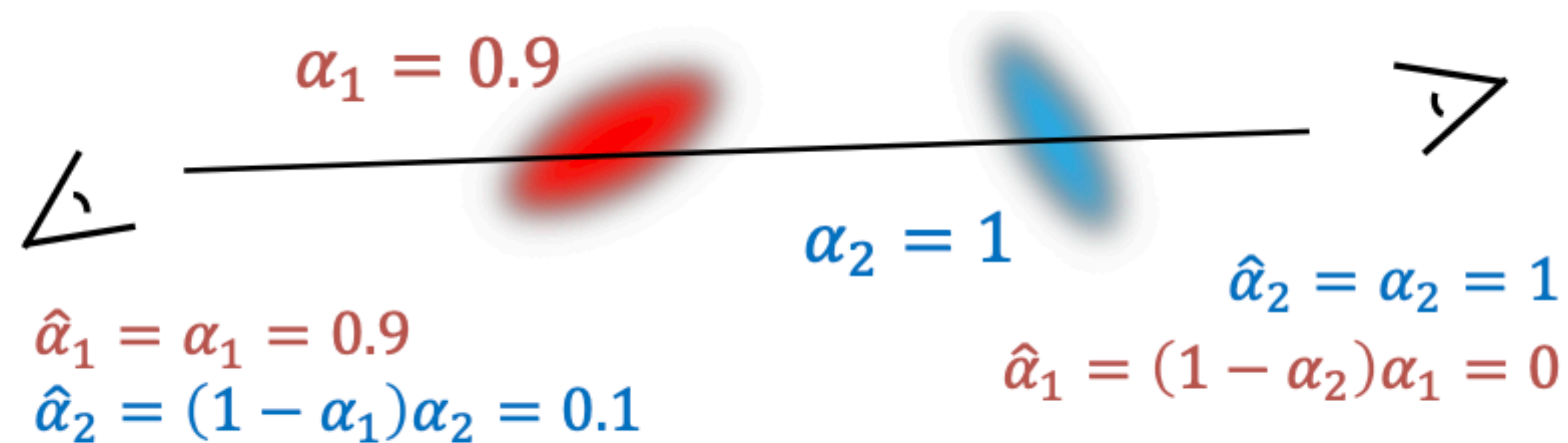


(b) w/ early termination

- ROP units are much fewer (about 100x) than shading units.
- ROP do not support early termination natively.

Why OIT rendering?

- The rendering order affects blending accuracy.
- OIT Rendering allows transparent geometry to be submitted in an arbitrary order.
 - Sorting free: eliminates the need for per-tile or per-Gaussian sorting.
 - Blending parallelism: decouple data dependency during blending.



Outline

- Background
 - What is 3DGS
 - 3DGS Rendering Pipeline
 - Problem: Why do we need OIT?
- **OIT methods in 3DGS**
 - Weighted Sum Rendering
 - Surfel Rendering
 - Stochastic Sampling

Weighted Sum Rendering

- Weighted Sum Rendering (WSR): an approximation of alpha blending.

- Alpha Blending:

$$C = \sum_i^N c_i \alpha_i \prod_{k=0}^{i-1} (1 - \alpha_k)$$

- Weighted Sum:

$$C = \text{Norm}(\sum_i^N c_i \alpha_i w(d_i))$$

w : Function that decrease with Gaussian depth d , to approximate transmittance.

Weighted Sum Rendering in 3DGS

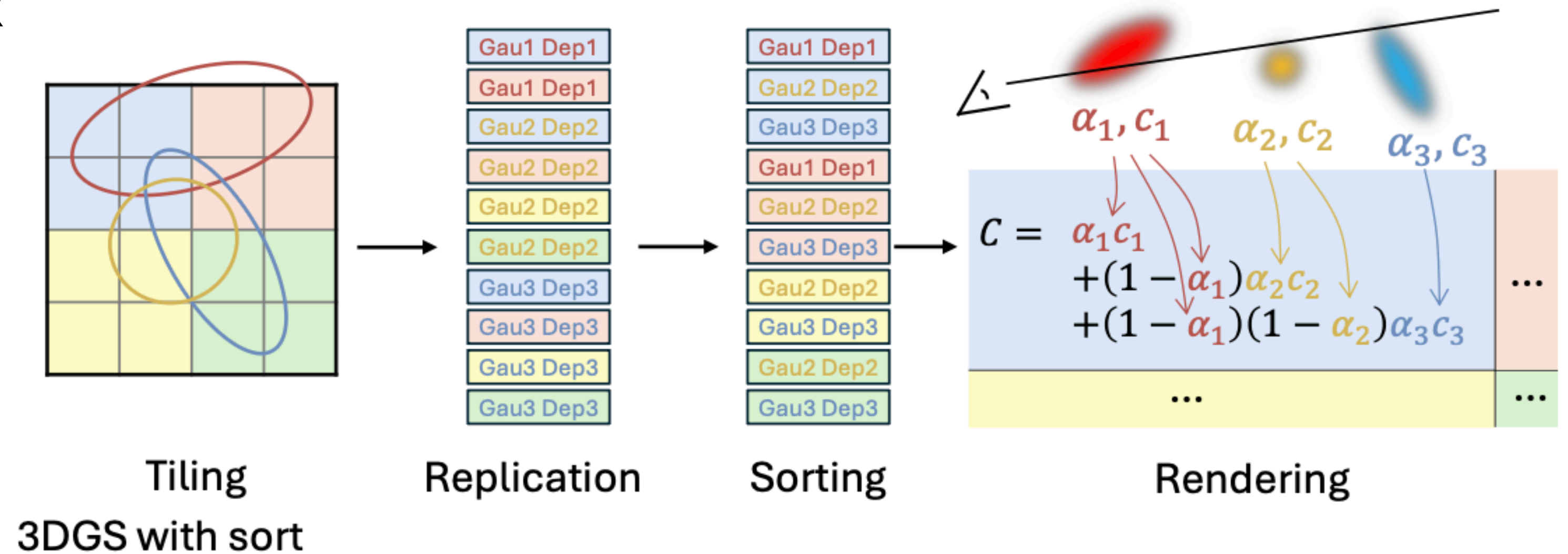
- (ICCV'25) Sort-free Gaussian Splatting via Weighted Sum Rendering

- Remove sorting stage via WSR

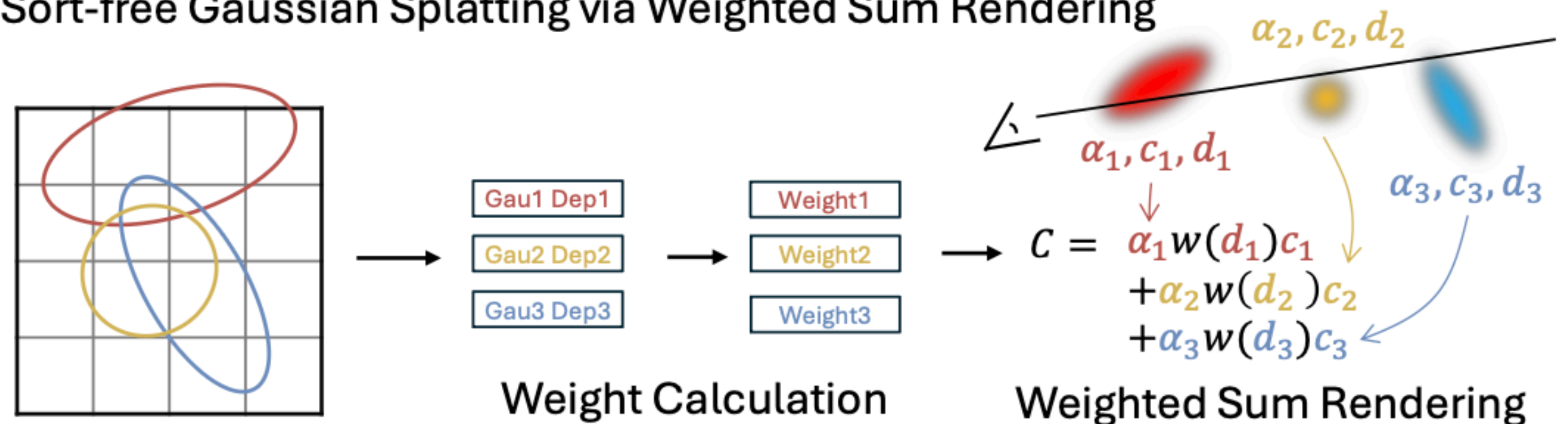
$$w(d_i) = \max(0, 1 - \frac{d_i}{\sigma})v_i$$

σ, v_i : Learnable param.

- Render via graphic pipeline



Sort-free Gaussian Splatting via Weighted Sum Rendering



Weighted Sum Rendering in 3DGS

- (ICCV'25) Sort-free Gaussian Splatting via Weighted Sum Rendering
 - Vulkan-based implementation results on **Snapdragon 8 Gen 3 (Qualcomm GPU)**

- Benefits
 - Model with fewer Gaussians
 - Removal of sorting step

(‘*’ indicates that mobile resources are exhausted)

Method	Task	Mip-NeRF360								
		bicycle	flowers	garden	stump	treehill	room	counter	kitchen	bonsai
3DGS-Compute	Pre-processing	12.84	7.05	10.15	6.94	7.53	3.99	3.02	4.46	3.12
	Lighting	4.57	3.26	5.19	3.43	3.37	1.84	1.55	3.07	1.40
	Sorting	246.65	122.68	210.35	180.23	139.98	70.83	50.57	89.96	50.63
	Rasterization	1239.12*	115.18	608.19*	265.27	147.08	140.16	85.94	177.01	73.43
	Total	1511.96	253.60	842.15	463.06	303.68	219.31	143.03	277.35	130.58
3DGS-Graphics	Pre-processing	13.27	7.82	13.73	10.46	8.01	3.40	2.62	4.09	2.62
	Lighting	5.32	3.66	5.67	3.68	3.80	2.00	1.61	3.55	1.51
	Sorting	40.26	24.79	38.25	31.09	25.47	11.23	9.47	13.76	9.17
	Rasterization	618.83*	246.47*	697.81*	344.72*	208.28*	32.50	28.22	52.84	30.86
	Total	678.08	283.11	755.89	390.33	245.95	49.46	42.25	74.56	44.47
Ours	Pre-processing	6.83	3.92	5.53	6.46	4.63	1.94	1.67	2.74	1.64
	Lighting	4.07	2.81	4.36	3.67	3.23	1.41	1.35	2.98	1.16
	Rasterization	67.84	28.30	57.84	57.25	52.99	45.44	32.82	68.51	37.52
	Total	78.99	35.28	67.90	67.63	61.11	49.02	34.06	74.47	40.63

Weighted Sum Rendering with Hardware Support

- (ISCA'26) Efficient 3D Gaussian Splatting with Axis-Shared Rasterization and OIT
 - Replace WSR function with a MLP
 - Design a reconfigurable PE array for MLP inference and rasterization.

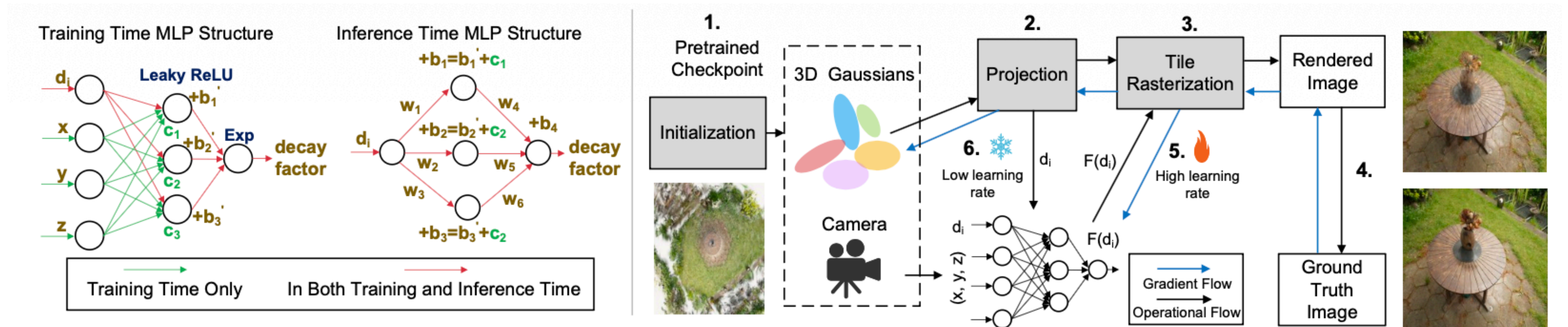


Fig. 7. Neural network structure (left) and the training framework (right).

Weighted Sum Rendering with Hardware Support

- (ISCA'26) Efficient 3D Gaussian Splatting with Axis-Shared Rasterization and OIT
- Replace WSR function with a MLP
- Design a reconfigurable PE array for MLP inference and rasterization.

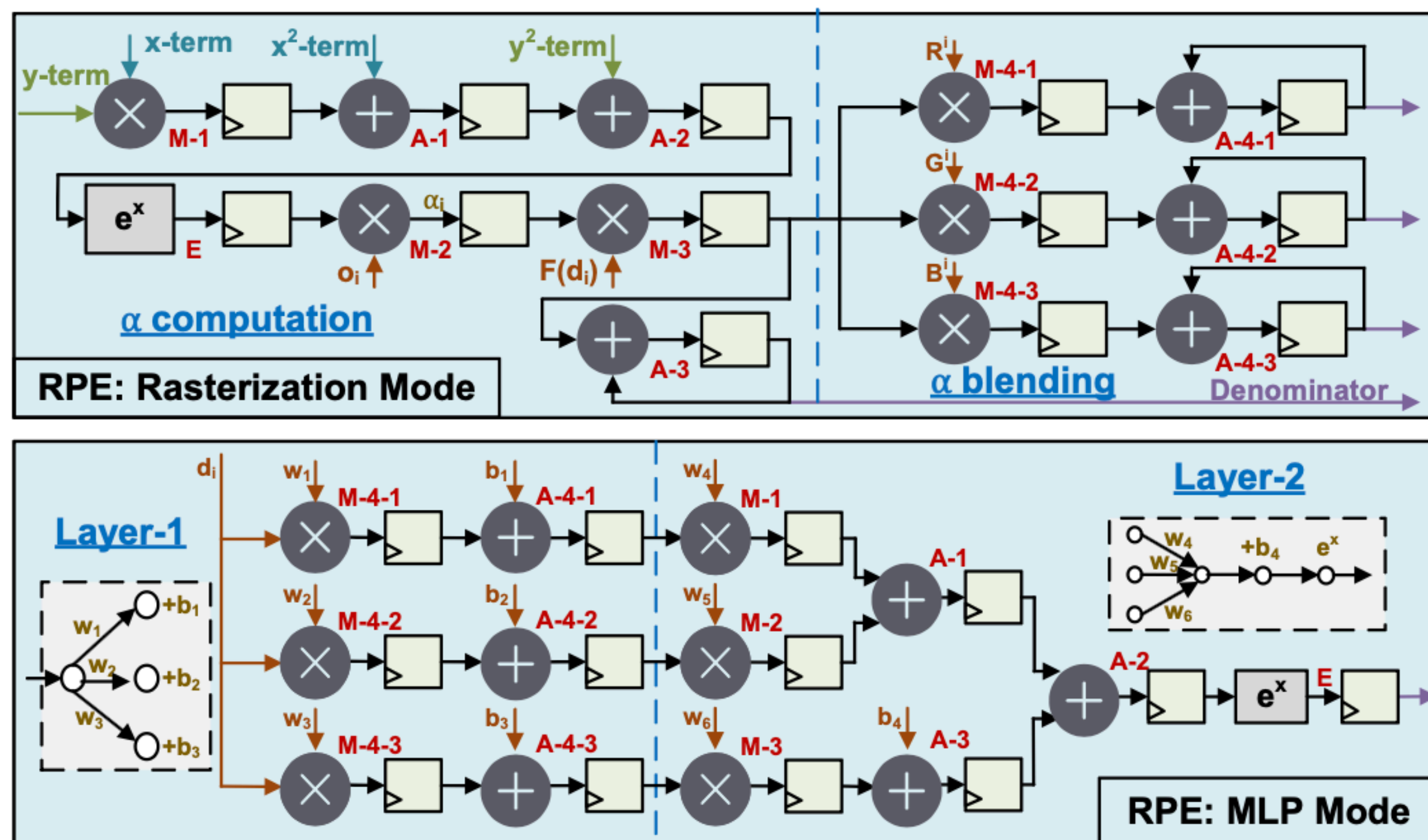


Fig. 9. Reconfigurable PE for rasterization and MLP modes.

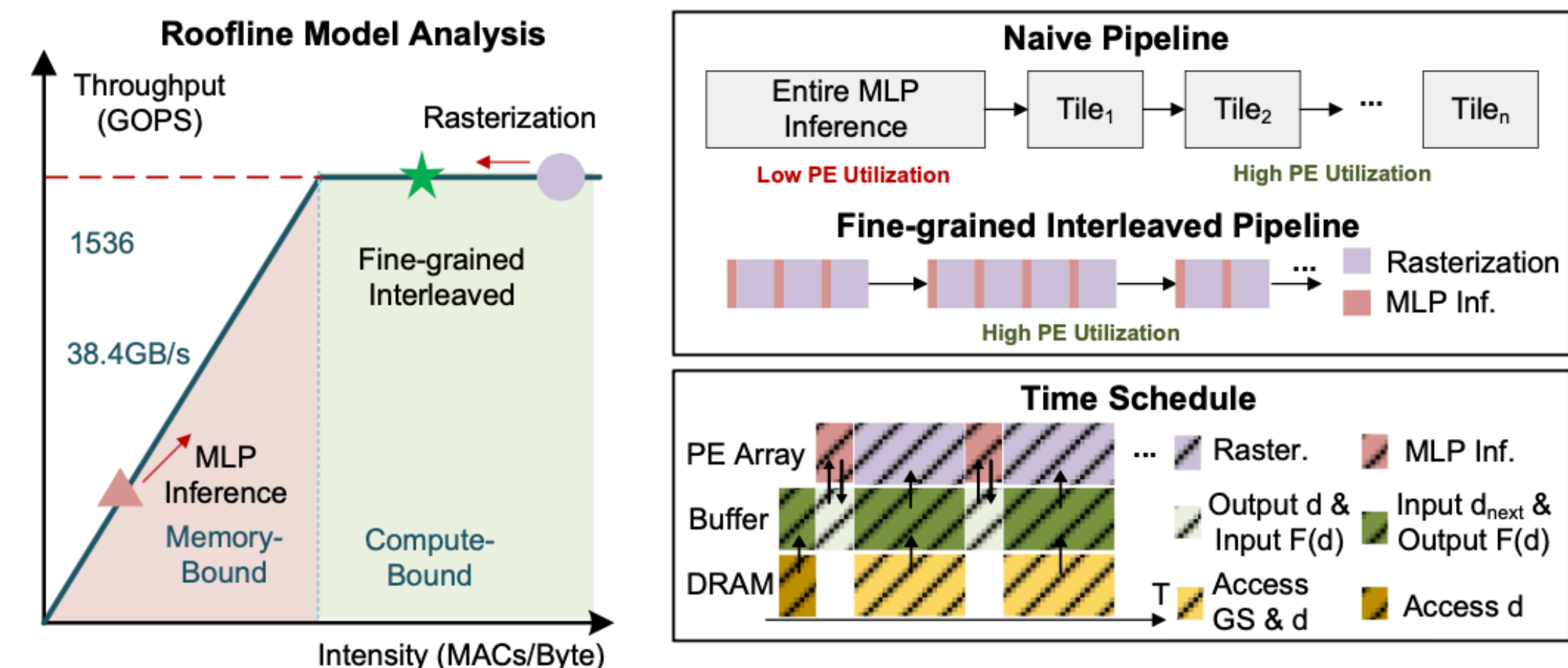
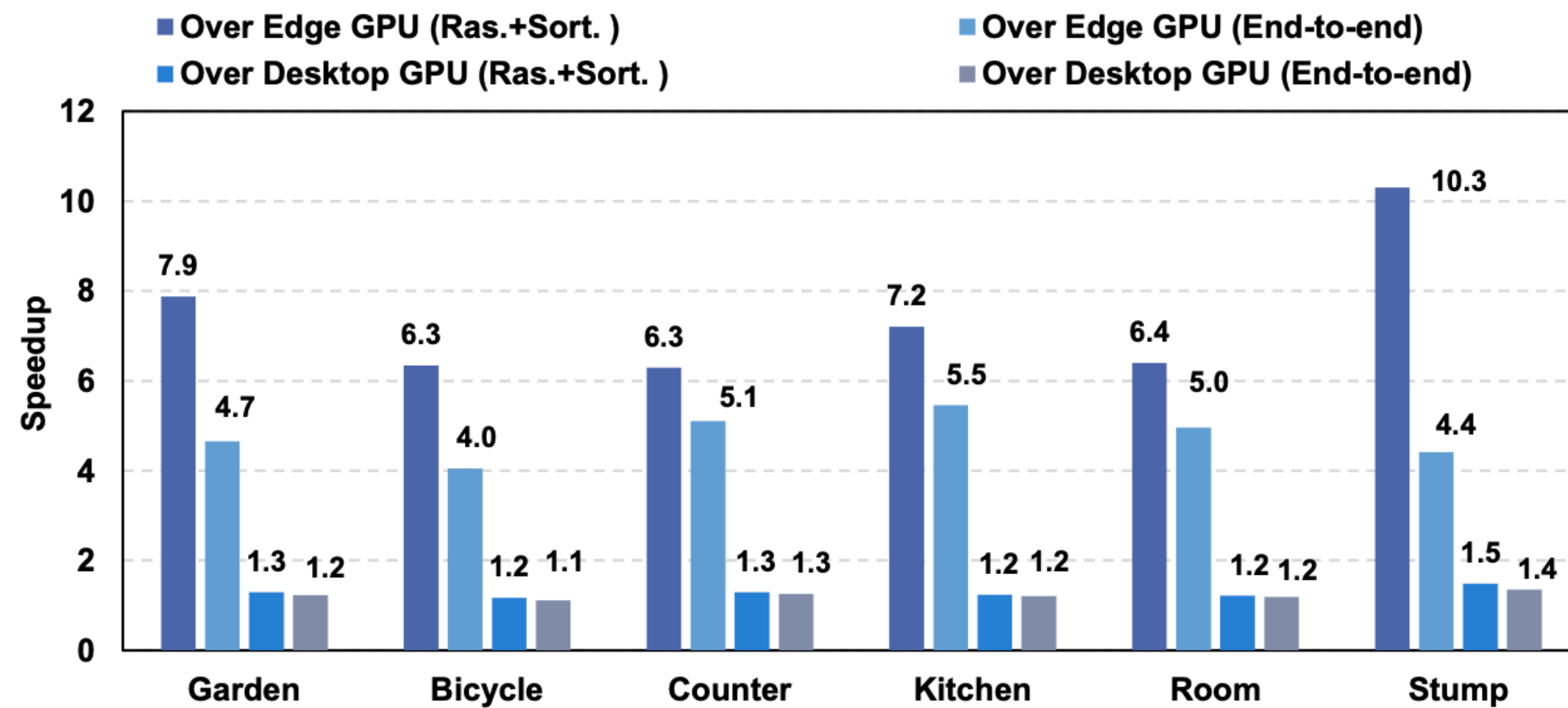


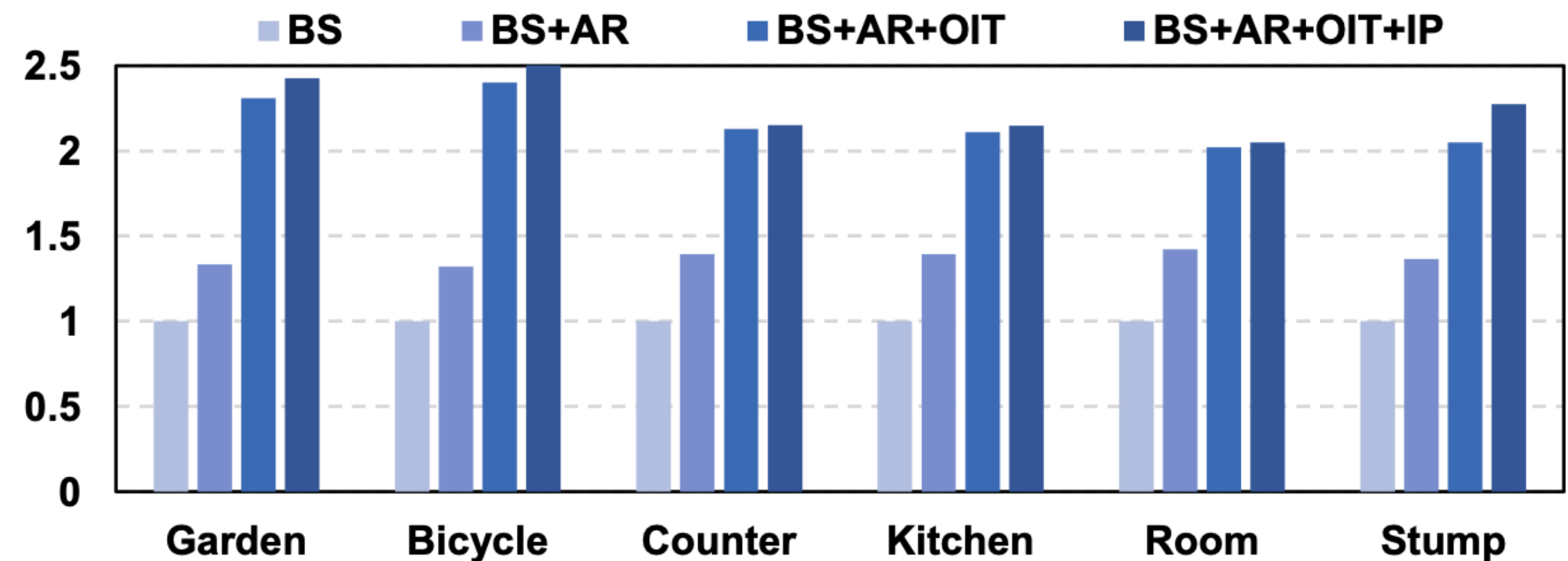
Fig. 10. Roofline model analysis and pipeline comparison.

Weighted Sum Rendering with Hardware Support

- (ISCA'26) Efficient 3D Gaussian Splatting with Axis-Shared Rasterization and OIT
 - Replace WSR function with a MLP
 - Design a reconfigurable PE array for MLP inference and rasterization.



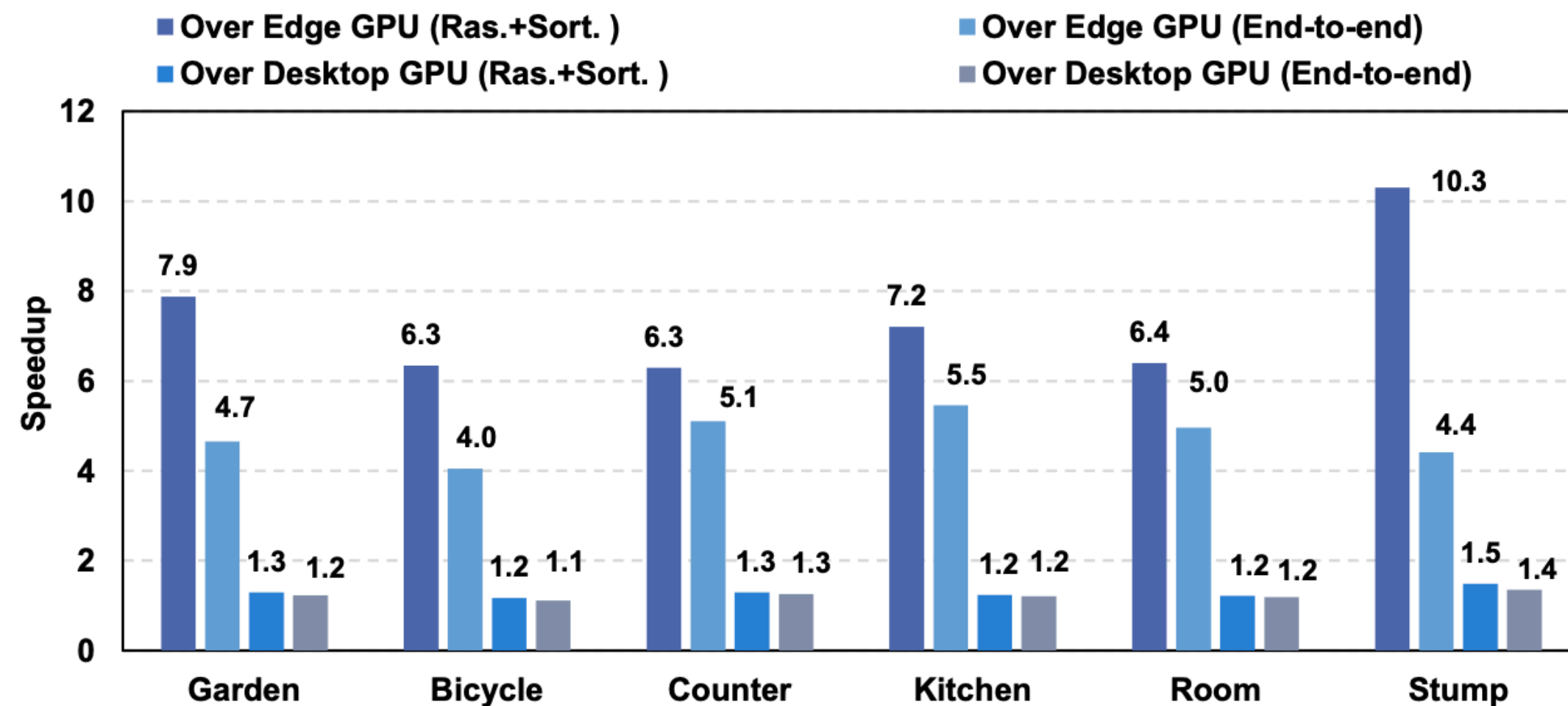
Speedup Evaluation



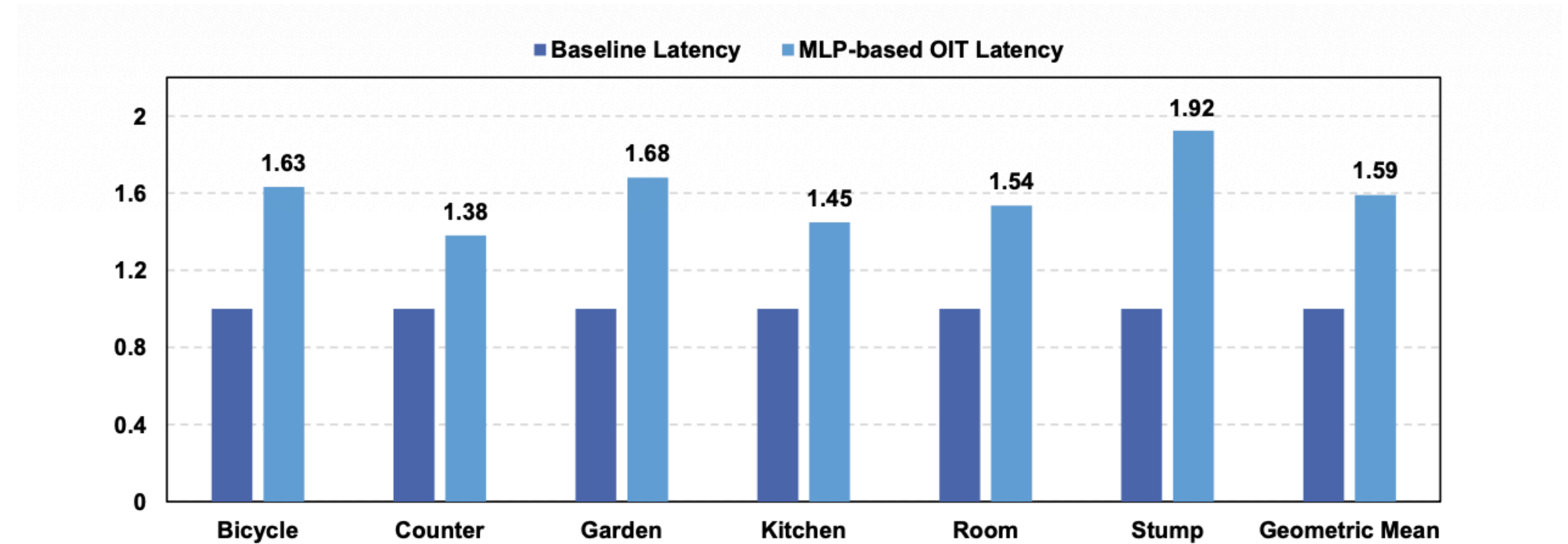
Hardware Ablation

Weighted Sum Rendering with Hardware Support

- (ISCA'26) Efficient 3D Gaussian Splatting with Axis-Shared Rasterization and OIT
 - Replace WSR function with a MLP
 - Design a reconfigurable PE array for MLP inference and rasterization.



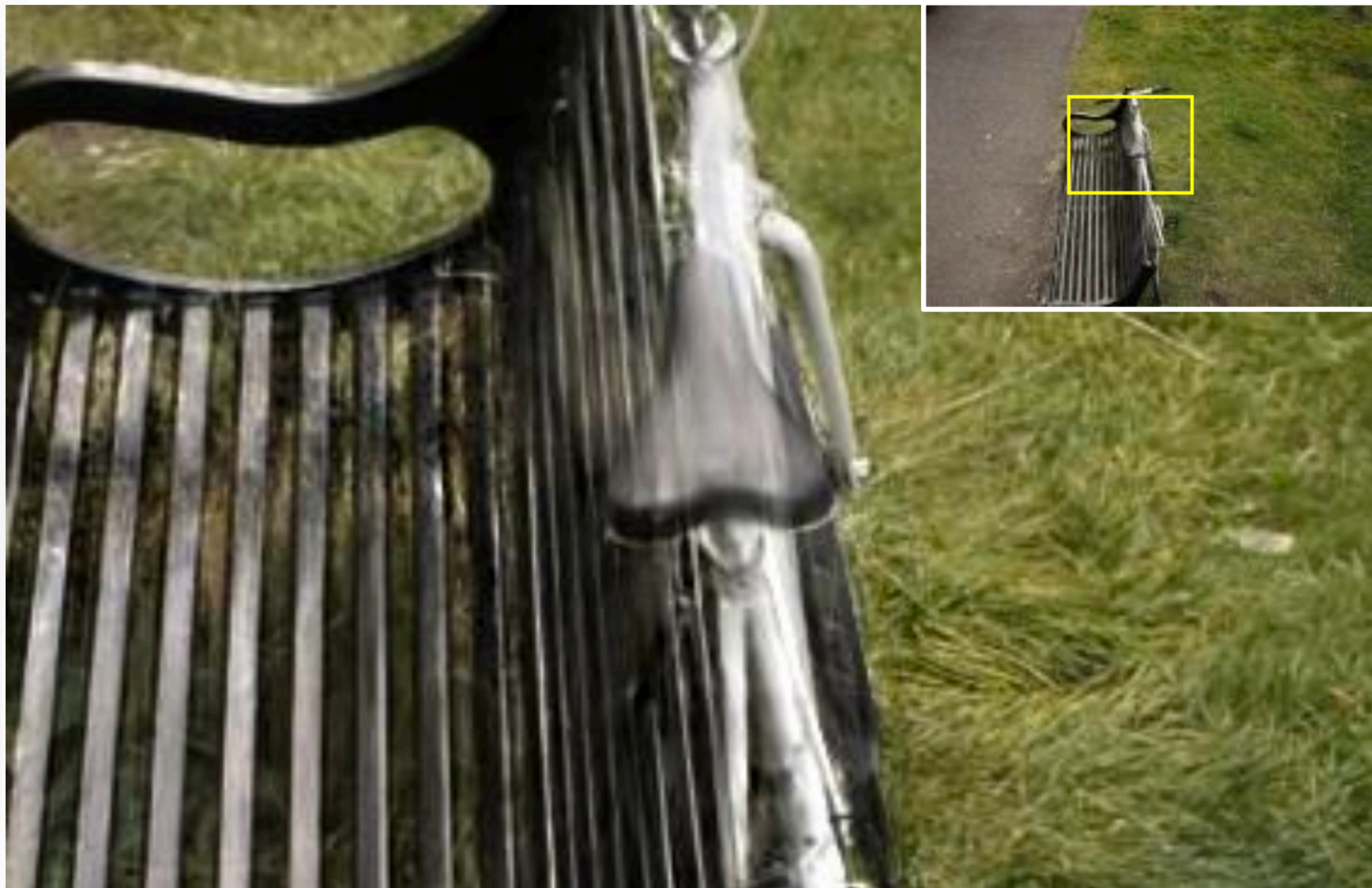
Speedup Evaluation



Comparison between sorting and MLP Inference

Limits in Weighted Sum Rendering

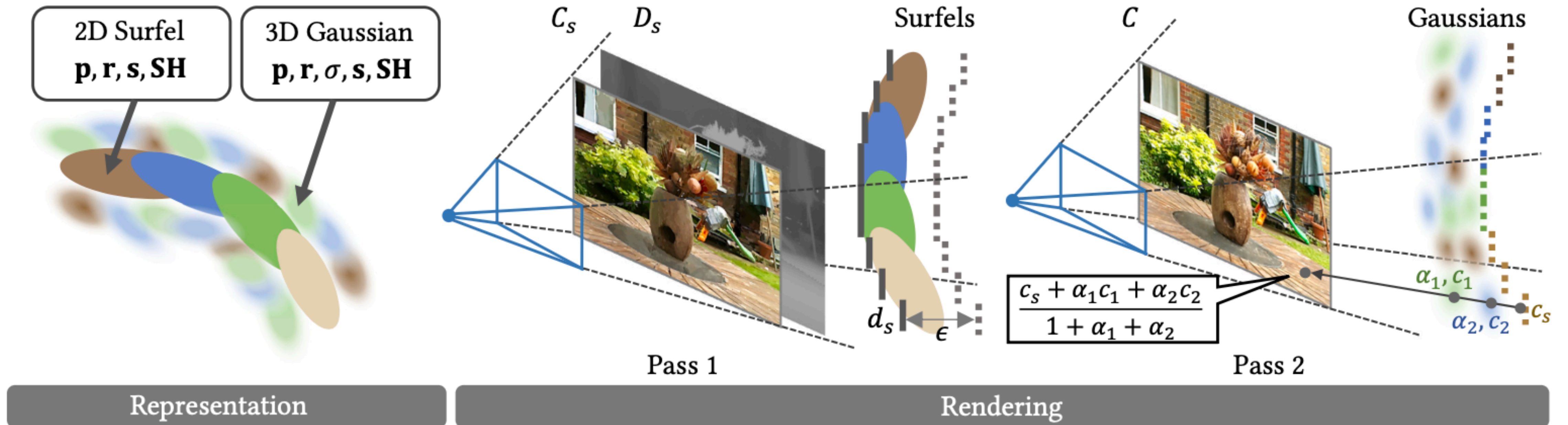
- Accuracy: incorrect rendering in some views.



- Blending overhead: WSR cannot reduce ROP pressure and limits speedup on desktop GPU.

Surfel Rendering

- Idea: Transparent Gaussian -> Opaque Surfel
- (Siggraph'25) When Gaussian Meets Surfel: Ultra-fast High-fidelity Radiance Field Rendering
 - Representation: opaque surfels and Gaussians surrounding them.
 - Opaque surfels: capture major portion of scene -> render with graphics pipeline
 - Translucent Gaussians: enhance scene with details -> render in WSR paradigm

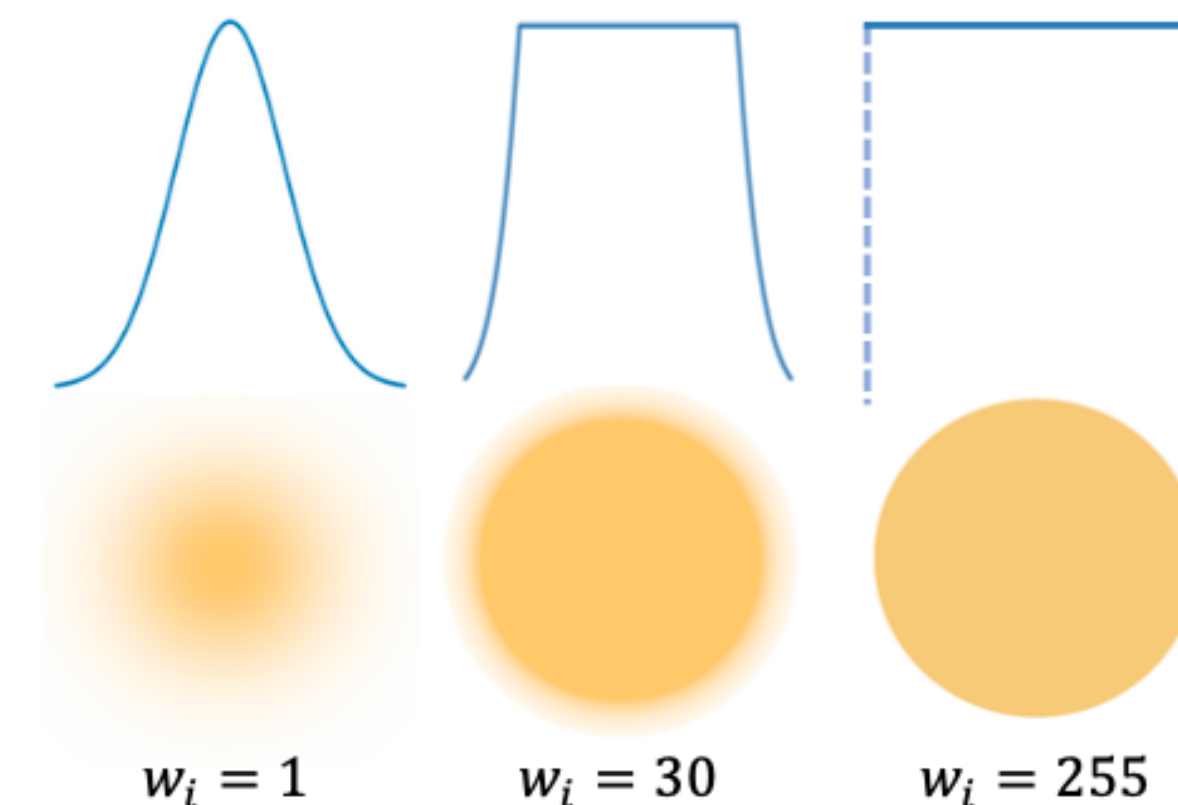


Surfel Rendering

- (Siggraph'25) When Gaussian Meets Surfel: Ultra-fast High-fidelity Radiance Field Rendering
 - Gaussian-Enhanced Surfel (GES) representation: surfels and Gaussians.
 - 2-pass Rendering by OpenGL: first opaque surfels, next translucent Gaussians.

	FPS (1080p)	FPS (2160p)	MEM	Train
3DGS	185	62	734	28
SortFreeGS*	321	168	506	42
StopThePop	167	55	830	40
MipSplat	131	43	1054	37
AdrGS	537	195	274	16
AbsGS	176	60	746	29
SpeedySplat	1140	369	78	14
C3DGS	139	47	49	36
2DGS	90	24	504	26
3D-GES	675	233	366	43
2D-GES	718	247	343	52
Mip-GES	640	213	394	49
Speedy-GES	1135	348	185	36
Compact-GES	300	128	47	39

- Surfel rendering achieves better speedup on models with more points.
- Limit: complex training process and more manually set hyperparameters.

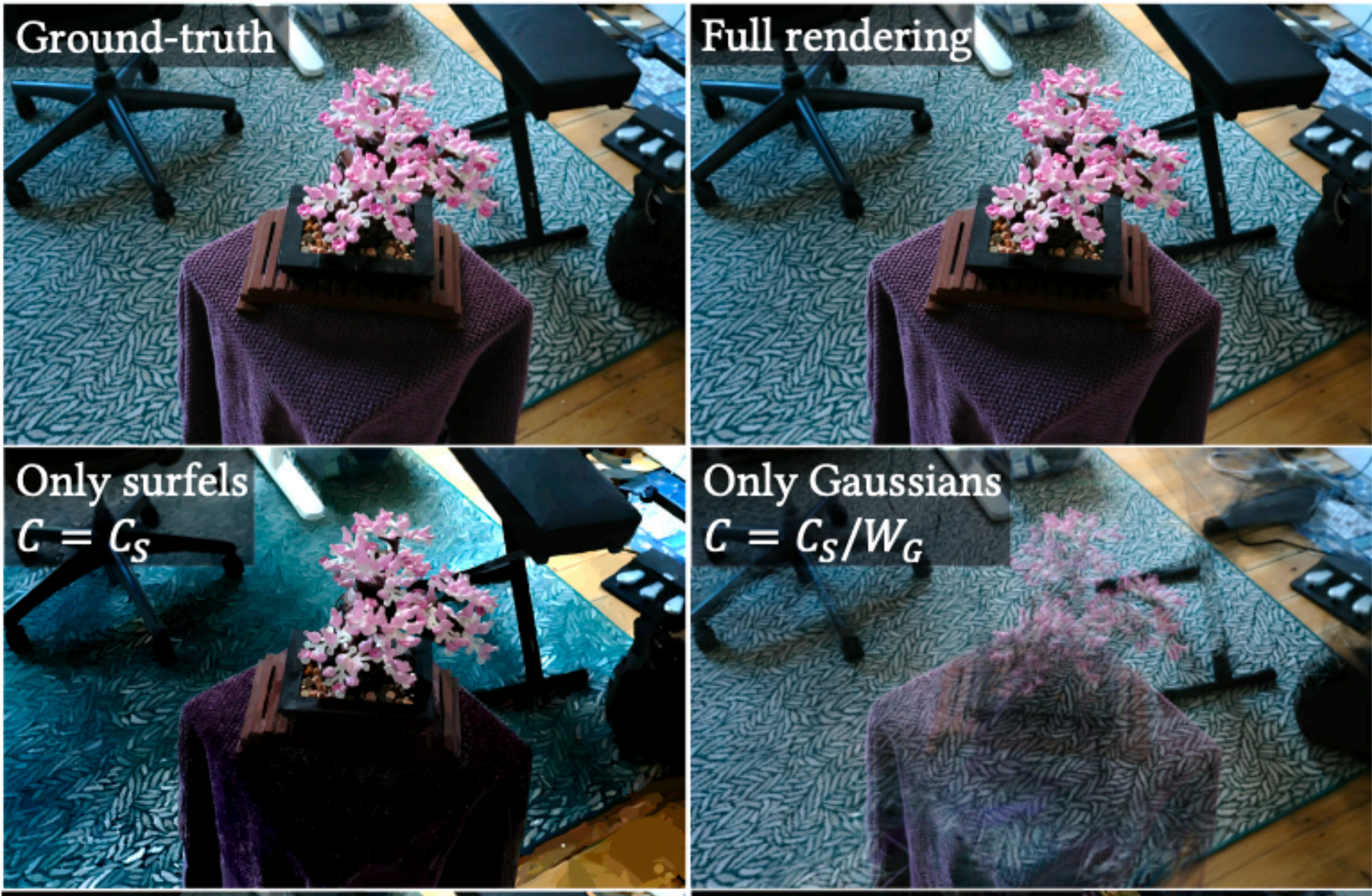


Surfel Rendering

- (Siggraph'25) When Gaussian Meets Surfel: Ultra-fast High-fidelity Radiance Field Rendering
 - Gaussian-Enhanced Surfel (GES) representation: surfels and Gaussians.
 - 2-pass rendering by OpenGL: first opaque surfels, next translucent Gaussians.

	FPS (1080p)	FPS (2160p)	MEM	Train
3DGS	185	62	734	28

	SSIM ↑	PSNR ↑	LPIPS ↓	FPS ↑
3D-GES	0.813	27.38	0.208	675
Single stage optimization	0.809	27.04	0.216	624
Only surfels	0.611	24.22	0.501	4771
Only Gaussians	0.774	26.02	0.272	705



Stochastic Transparency

- An example of Stochastic sampling: global illumination
 - The brightness of a single point comes from the integral of light incident in all direction.
 - Monte Carlo estimation is used to approximate integration results.
- (ICCV'25) StochasticSplats: Stochastic Rasterization for Sorting-Free 3DGS
 - Approximate blending result through Monte Carlo estimator.
 - Achieve Stochastic sampling through standard graphic pipeline.

$$C = \sum_i^N c_i P(i)$$

$$\text{where } P(i) = \alpha_i \prod_{k=0}^{i-1} (1 - \alpha_k)$$

C is the expectation of stochastic variable c_i where $i \sim P(i)$

```
 $z \leftarrow \infty, C \leftarrow c_{background}$ 
for each Gaussian  $i$  in unsorted order do
  Compute  $\alpha_i$  and depth  $z_i$ 
  Sample  $u \sim \mathcal{U}(0, 1)$ 
  if  $u < \alpha_i$  and  $z_i < z$  then
     $C \leftarrow c_i$ 
     $z \leftarrow z_i$ 
  end if
end for
```

Listing 1. Pseudocode for stochastic transparency.

Stochastic Transparency

- How to integrate stochastic sampling with graphic pipeline?
 - Early-z: automatically pick Gaussians by depth test and sample in fragment shader.
 - MSAA: achieve multiple sampling in a single fragment shading.

	3DGS-CUDA	3DGS-OpenGL	StopThePop		Ours-OpenGL (SPP)				
				1	2	4	8	16	
T1000	65.08	100.28	75.91	16.23	18.95	24.71	37.51	61.25	
RTX3090	8.14	32.15	9.48	3.25	4.18	6.42	15.31	18.48	
RTX4090	5.60	20.70	-	1.85	2.05	2.86	6.71	8.00	

	3DGS	StopThePop			Ours (SPP)			
			1	4	16	64	256	1024
PSNR	28.99	28.79	17.95	22.81	26.25	27.93	28.50	28.66
SSIM	0.869	0.870	0.285	0.512	0.714	0.819	0.856	0.867
LPIPS	0.185	0.181	0.611	0.493	0.351	0.235	0.178	0.168

- In single-view rendering, ST requires more SPP.
- In video rendering, Temporal Anti-Aliasing (TAA) can reduce SPP.
- Promising for 3DGS ray tracing.



Figure 9. **Temporal anti-aliasing** – StochasticSplats at 1 SPP without (left) and with (right) TAA after moving along a trajectory for 70 frames. TAA introduces a negligible latency increase.

Outline

- Background
 - What is 3DGS?
 - Tile-based 3DGS rendering pipeline.
 - Problem: why do we need OIT?
- OIT methods in 3DGS
 - Weighted Sum Rendering
 - Surfel Rendering
 - Stochastic Transparency